

Практика 4. Введение в Haskell

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Синтаксис и семантика лямбда-исчисления
- ▶ Просто-типизированное лямбда-исчисление
- + Базовый синтаксис Haskell
- + Базовые типы
- + Группировка объявлений: `let` и `where`
- + Операторы и сечения

Содержание

1. Рекурсия и аккумуляторы
2. Полиморфные функции
3. Инфраструктура Haskell (⊕)
4. Материалы (☀)

Содержание

1. Рекурсия и аккумуляторы
2. Полиморфные функции
3. Инфраструктура Haskell (⊕)
4. Материалы (☀)

Задачи на рекурсию

todo Реализуйте факториал

todo Реализуйте примитивную рекурсию



Математика

$$n! = \begin{cases} 1, & n \leq 1 \\ n * (n - 1)!, & otherwise \end{cases}$$

Haskell

```
fac n
  | n <= 1 = 1
  | otherwise = n * fac (n - 1)
```



```
rec :: (Int -> a -> a) -> a -> Int -> a
rec f ini n = if n <= 0 then ini else f (n - 1) (rec f ini (n - 1))
```

Техника аккумулярующего параметра

todo Какая сложность по памяти у рекурсивного факториала?



Линейная:

```
fac 4 -> 4 * fac 3 -> 4 * (3 * fac 2)
-> 4 * (3 * (2 * fac 1)) -> 4 * (3 * (2 * 1)) -> 24
```

- Будем накапливать промежуточные результаты в параметре-аккумуляторе

```
fac' n = go n (n - 1)
  where
    go acc n'
      | n' <= 0 = acc
      | otherwise = go (acc * n') (n' - 1)
```

todo Проредуцируйте пример `fac' 4`



```
fac' 4 -> go 4 3 -> go (4 * 3) 2 -> go ((4 * 3) * 2) 1
-> go (((4 * 3) * 2) * 1) 0 -> ((4 * 3) * 2) * 1 -> 24
```

- Выражения можно сокращать сразу! Константа по памяти

Злополучный Фибоначчи

- Реализация по определению имеет экспоненциальную сложность

todo Реализуйте эффективное вычисление n-го числа Фибоначчи



```
fib :: Int -> Int
```

```
fib n = go 1 1 n
```

```
  where
```

```
    go prev curr n
```

```
      | n <= 1 = prev
```

```
      | otherwise = go curr (prev + curr) (n - 1)
```

Содержание

1. Рекурсия и аккумуляторы
2. Полиморфные функции
3. Инфраструктура Haskell (⊕)
4. Материалы (☀)

Полиморфные функции

- ▶ Типы с маленькой буквы — типовые переменные
- ▶ Могут принимать разные значения в зависимости от контекста

```
todo ghci> :t id
todo ghci> :t id "something here"
todo ghci> :t const
todo ghci> :t const id
todo ghci> :t flip const
todo ghci> :t uncurry (flip const)
⚡ id :: a -> a
⚡ id "something here" :: String
⚡ const :: a -> b -> a
⚡ const id :: b -> a -> a
⚡ flip const :: b -> a -> a
⚡ uncurry (flip const) :: (b, a) -> a
```

Конструирование реализаций

```

($) :: (a -> b) -> a -> b
(&) :: a -> (a -> b) -> b
(.) :: (b -> c) -> (a -> b) -> a -> c
f :: Int -> a -> a
g :: Int -> Double
h :: Int -> Double -> Int

```

`todo` Запишите через доллар `example x = f 1 $ g $ h 2 x`

`todo` Запишите через композицию

`todo` Запишите в бесточечном стиле



```

example x = f 1 $ g $ h 2 x
example x = (f 1 . g . h 2) x
example = f 1 . g . h 2

```

Больше полиморфных функций¹

`todo` Реализуйте функцию, применяющую функцию к первой компоненте пары

`todo` Прибавьте единицу к первой компоненте `((Int, a), b)`

⚡ `first :: (a -> a') -> (a, b) -> (a', b)`

`first f (x, y) = (f x, y)`

⚡ `first . first :: (a -> a') -> ((a, b), c) -> ((a', b), c)`

`(first . first) (+1) value :: ((Int, b), c)`

¹<http://conal.net/blog/posts/semantic-editor-combinators>

Содержание

1. Рекурсия и аккумуляторы
2. Полиморфные функции
3. Инфраструктура Haskell (⊕)
4. Материалы (☀)

Компилятор GHC

- ▶ Единственный компилятор Haskell
- ▶ Язык развивается в основном расширениями компилятора
 - ▶ Подключение прагмой в начале файла `{-# LANGUAGE ... #-}`
 - ▶ Или ключами компиляции (в конфигах проекта)
 - ▶ Добавляют синтаксический сахар (`LambdaCase`, `ImplicitParams`)
 - ▶ Обогащают систему типов (`RankNTypes`)
 - ▶ Добавляют фичи (`MultiParamTypeClasses`, `GADTs`)
 - ▶ Добавляют экспериментальные фичи (`LinearTypes`)
- ▶ https://ghc.gitlab.haskell.org/ghc/doc/users_guide/index.html

Системы сборки

- ▶ По конфигурации проекта правильно вызывают компилятор

Cabal Common Architecture for Building Applications and Libraries

- ▶ Система сборки + библиотека

Stack Более новая система, надстройка над cabal

- ▶ Сама скачает компилятор нужной версии
- ▶ `stack new my-project; cd my-project`
- ▶ `stack run` — запустить `main`
- ▶ `stack build` — собрать проект
- ▶ `stack test` — запустить тесты
- ▶ `stack ghci` — REPL с загруженным проектом

REPL, GHCi

REPL (read-eval-print-loop)

Интерактивное окружение, исполняющее вводимые в него высказывания и тут же демонстрирующее результат.

- ▶ Компилятор работает долго, а GHCi — быстро
- ▶ Пишем код, загружаем в интерпретатор: `stack ghci`
- ▶ Основные возможности

Исполнить выражение

`:r (reload)` Перезагрузить интерпретатор

`:t (type)` Посмотреть тип выражения

`:i (info)` Посмотреть документацию функции

`:set` Установить расширения языка и флаги компиляции

Типовые дыры в Haskell

В определении функции можно ставить подчеркивания:

```
plus' :: Int -> Int -> Int
plus' x y = x + _
```

```
Found hole: _ :: Int
```

```
In the second argument of (+), namely _
```

```
In the expression: x + _
```

```
In an equation for plus': plus' x y = x + _
```

```
Relevant bindings include
```

```
y :: Int
```

```
(bound at /home/user/fp/examples/src/Lib.hs:9:9)
```

```
x :: Int
```

```
(bound at /home/user/fp/examples/src/Lib.hs:9:7)
```

```
plus' :: Int -> Int -> Int
```

```
(bound at /home/user/fp/examples/src/Lib.hs:9:1)
```

Другие тулы

- ▶ Линтер (`hlint`) помогает находить ошибки в программах
- ▶ Очень помогает в обучении

NB На этом курсе обязателен к использованию!

- ▶ Форматтер автоматически форматирует код: `brittany`, `ormolu`...
- ▶ Обязателен к использованию во многих языках: экономит время и повышает консистентность кода
- ▶ В Haskell не всегда удовлетворительно работает

NB На этом курсе запрещён! (для упрощения ревью)

LSP

- ▶ LSP — Language Server Protocol — это протокол взаимодействия между движками IDE и текстовыми редакторами
- ▶ Есть language server для большинства языков
- ▶ [Есть для Haskell](#), работает *только* для той же версии ghc
- ▶ Большинство редакторов кода поддерживает LSP и с его помощью почти превращается в IDE

IDE для Haskell на примере vscode

- ▶ Устанавливаете в систему `haskell-language-server`
- ▶ Устанавливаете в систему `hlint`
- ▶ Ставите плагины: [Haskell \(может поставить и остальные\)](#), [Haskell Syntax](#), [haskell-linter](#)

Содержание

1. Рекурсия и аккумуляторы
2. Полиморфные функции
3. Инфраструктура Haskell (⊕)
4. Материалы (☀)

Полиморфные функции (*)

- * То, что типизация остаётся валидной после подстановки в тип — теорема метатеории

Theorem (Лемма подстановки типа для $\lambda_{\rightarrow}$)

$\Gamma \vdash M : \sigma \Rightarrow [\alpha \mapsto \tau] \Gamma \vdash M : [\alpha \mapsto \tau] \sigma$ ($\lambda_{\rightarrow}$ по Карри)

$\Gamma \vdash M : \sigma \Rightarrow [\alpha \mapsto \tau] \Gamma \vdash [\alpha \mapsto \tau] M : [\alpha \mapsto \tau] \sigma$ ($\lambda_{\rightarrow}$ по Чёрчу)

- ▶ Если мы в языке разрешим приписывать типы термам, мы потеряем гибкость:

$\mathbf{K} : \alpha \rightarrow \beta \rightarrow \alpha$

$\mathbf{K} \ 1 : \text{cannot match } \alpha \text{ with } \mathbf{Int}$

- ▶ Введём в теорию абстракцию на типах и аппликацию термов к типам!

$\mathbf{K} : \forall \alpha \beta. \alpha \rightarrow \beta \rightarrow \alpha$

$\mathbf{K} \ @\mathbf{Int} : \forall \beta. \mathbf{Int} \rightarrow \beta \rightarrow \mathbf{Int}$

$\mathbf{K} \ @\mathbf{Int} \ @\mathbf{Float} \ 1 : \mathbf{Float} \rightarrow \mathbf{Int}$

- ▶ Теперь выбор конкретного типа происходит на стороне пользователя функции
- ▶ Такие функции называются **полиморфными** — умеют работать с различными типами

Полиморфные функции в Haskell (*)

- ▶ Haskell неявно приписывает forall по всем типовым переменным:

```
ghci> :set -fprint-explicit-foralls
ghci> :t const
const :: forall {a} {b}. a -> b -> a
```

- ▶ Типовые переменные всегда начинаются с маленькой буквы (`a`), в отличие от типовых констант (`Int` — конкретный тип)
- ▶ Типовые применения компилятор выставляет автоматически неявно
- ▶ Иногда бывает полезно типовые применения указывать вручную (`TypeApplications`)

- ▶

```
ghci> :set -XTypeApplications
ghci> :t const @Int
const @Int :: forall {b}. Int -> b -> Int
```

- ▶ В C++-like языках синтаксис примерно такой:

```
fun <A, B> const(x: A, y: B): A
const<Int, Float>(1, 2)
```

Что посмотреть в транспорте

- ▶ Vitaly Bragilevsky - The clear path to Haskell complexities
- ▶ Podlodka #211 – Haskell | Александр Вершилов
- ▶ Property-based testing в Haskell — так тестируются наши домашки

- ▶ [Hoogle](#) — типизированный поиск по документации библиотек
- ▶ Отладка принтами: [Debug.Trace](#)
- ▶ [GHC docs](#) — про компилятор и расширения языка
- ▶ [Typeclassopedia](#)
- ▶ [Stack build system](#)
- ▶ [The Haskell Execution Model](#)
- ▶ [I know kung fu: learning STG by example](#)

- ▶ Will Kurt "Get Programming with Haskell"
- ▶ Две части курса Дениса Николаевича на степике: [раз](#) и [два](#)
- ▶ (youtube) Лекции Дениса Николаевича на МКН
- ▶ Vitaly Bragilevsky "Haskell in depth"
- ▶ (youtube) Компилятор GHC языка Haskell: теория языков программирования в работе
- ▶ (youtube) Continuation-passing style, defunctionalization, and associativity.